Configurar Kubernetes

- Un Master
- Un Worker solamente

Requisito. Haber configurado acceso ssh via clave entre los nodos

configurar-ssh-via-clave

previo - Inventario de equipos

Con linux como hipervisor:

- Listar mis máquinas:

```
virsh list --all
```

- Clonar una VM:

```
virt-clone --original rocky9-base --name rocky9-clone --auto-clone
```

- Conocer la IP de las VM que se encuentran en ejecución

```
virsh net-dhcp-leases default
```

- Eliminar una VM

```
virsh undefine rocky9-clone --remove-all-storage --nvram
```

inventario en mi caso:

Dos rorcy9 con las siguientes IP

```
root@pc-bda-mqr:~# virsh net-dhcp-leases default
 Expiry Time           MAC address         Protocol   IP address           Hostname       Client ID or DUID
-------------------------------------------------------------------------------------------------------------
 2026-11-05 01:14:18   52:54:00:45:59:af   ipv4       192.168.122.215/24   k8s-master01   01:52:54:00:45:59:af
 2026-11-05 01:14:22   52:54:00:c4:4e:29   ipv4       192.168.122.196/24   k8s-worker01   01:52:54:00:c4:4e:29
```

1 - Configurar hostnames

En contra de mi voluntad, trabajaré con root, para no complicar las cosas

1.1 - En el master (192.168.122.215)

1. Configurar hostname de manera inmediata mediante la herramienta

```
hostamectl set hostname "k8s-master1"
```

O también es válido manualmente editar **/etc/hostname** y colocar

```
ks8-master1
```

2. Agregar identificador DNS para los demás nodos **/etc/hostname**

```
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.0.1   ks8-master01
192.168.122.40  ks8-master01
192.168.122.120 k8s-worker01
```

Las importantes son las últimas tres entradas solamente

1.2 - En el worker 1 (192.168.122.196)

3. Configurar hostname de manera inmediata mediante la herramienta

```
hostamectl set hostname "k8s-worker1"
```

O también es válido manualmente editar **/etc/hostname** y colocar

```
ks8-master1
```

4. Agregar identificador DNS para los demás nodos **/etc/hostname**

```
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.0.1 k8s-worker01
192.168.122.215 k8s-master01
192.168.122.196 k8s-worker01
```

2 - Deshabilitar swap en cada nodo

apagando la swap

```
swapoff -a
```

deshabiliatar la entrada de swap en **/etc/fstab**

se encontraba así

```
/dev/mapper/rl-root     /                       ext4    defaults        1 1
UUID=4c93ab3b-2392-4fb7-a0fd-ac0d4f865e1a /boot                   xfs     defaults        0 0
/dev/mapper/rl-home     /home                   ext4    defaults        1 2
/dev/mapper/rl-swap     none                    swap    defaults        0 0
```

Se elimina la entrada de swap, quedando así

```
/dev/mapper/rl-root     /                       ext4    defaults        1 1
UUID=4c93ab3b-2392-4fb7-a0fd-ac0d4f865e1a /boot                   xfs     defaults        0 0
/dev/mapper/rl-home     /home                   ext4    defaults        1 2
```

verificar que swap está en cero

```
free
```

3 - Ajustar SELlinux y las reglas de firewall

3.1 - Deshabilitar selinux

de manera inmediata

```
setenforce 0
```

y de manera permanente editando **/etc/sysconfig/selinux**

orignalmente en modo enforcing

```
#    grubby --update-kernel ALL --remove-args selinux
#
SELINUX=enforcing
# SELINUXTYPE= can take one of these three values:
#     targeted - Targeted processes are protected,
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

para quedar así

```
SELINUX=permissive
# SELINUXTYPE= can take one of these three values:
#     targeted - Targeted processes are protected,
#     mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

comprobar

```
getenforce
```

debe salir: permisive

3.2 - Abrir los puertos en master01 (192.168.122.215)

conectarse a master01

```
ssh master01
```

Abrir los siguientes puertos con firewall-cmd

```
firewall-cmd --permanent --add-port={6443,2379,2380,10250,10251,10252,10259,179}/tcp
firewall-cmd --permanent --add-port=4789/udp
firewall-cmd --reload
```

3.3 - Abrir los puertos en worker01 (192.168.122.196)

conectarse a master01

```
ssh worker01
```

Abrir los siguientes puertos con firewall-cmd

```
firewall-cmd --permanent --add-port={179,10250,30000-32767}/tcp
firewall-cmd --permanent --add-port=4789/udp
```

4 - Parámetros del Kernel

modprobe: Es una utilidad para modificar (cargar/descargar) parámetro del Kernel de linux

4.1 - Cargar lo parámetros (overlay y br_netfilter)

(en master01)

Usando modprobe

```
modprobe overlay
modprobe br_netfilter
```

Para hacerlo permanente editar el archivo **/etc/modules-load.d/containerd.conf**

y colocar el siguiente contenido

```
overlay
br_netfilter
```

o bien ejecutar el siguiente comando para inyectarlo usando un [here document](How to create documents with Bash scripts)

```
cat <<EOF > /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

(Realizar los mismos pasos en worker01)

4.2 - Parámetros de manejo IP

(En master01)

gemini ha recomendado usar el archivo /etc/sysctl.d/99-kubernetes-cri.conf en lugar de /etc/syctl.d/k8s.conf , le harémos caso.

Crear el archivo de configuración **/etc/sysctl.d/99-kubernetes-cri.conf**

y colocar los siguientes parámetros

```
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

o bien ejecutar este here document que hace lo mismo

```
cat <<EOF > /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
```

Aplicar los cambios

```
sudo sysctl --system
```

4.3 - Comprobar

Comprobar que haya funcionado

```
sysctl net.ipv4.ip_forward net.bridge.bridge-nf-call-iptables net.bridge.bridge-nf-call-ip6tables
```

ha de salir algo asi:

```
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

Hacer lo mismo en (worker01)

5 - Instalar Containerd

Realizar en ambos rockys (master01 y worker01)

Agregar rpositorio de docker comunity edition

```
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

instalar containerd

```
sudo dnf install containerd.io -y
```

Generar el archivo de configiración, tomando como base la configuración por defecto

```
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml
```

opcional . verificarlo con *cat /etc/containerd/config.toml*

Iniciar / habilitar y verificar servicio

iniciar

```
sudo systemctl start containerd
sudo systemctl status containerd
systemctl enable containerd
```

3 - Instalar herramientas de kubernetes

Agregar el repositorio de kubernetes para tener acceso a los paquetes kubeadm, kubectl, y kubelet

Ejecutar el siguiente here document que crea el archivo de respositorio

```
cat << 'EOF' > /etc/yum.repos.d/kubernetes.repo
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.28/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
```

instalar los paquetes

```
dnf install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
```

Habilitar el servicio kubelet

```
systemctl enable --now kubelet
```

Sí en este momento se verifica el estatus de kubelet, mostrará un error, es normal, aun no se ha inicado el cluster



7 - Instalar el cluster de Kubernetes

En master01 (192.168.122.215)

Ejecutar el comando para iniciar el cluster de kubernetes

```
kubeadm init --control-plane-endpoint=k8s-master01 --pod-network-cidr=10.244.0.0/16
```

mostrará que se ha iniciado el cluster y hasta abajo el comando con el cual podemos agregar los workers al cluster.

Estos tokens pueden volver a generarse en cualquier momento con el comando

```
kubeadm token create --print-join-command
```

Configurar el directorio **.kube** para root en **master01**

```
mkdir -p /root/.kube
cp -i /etc/kubernetes/admin.conf /root/.kube/config
chown root:root /root/.kube/config
export KUBECONFIG=/etc/kubernetes/admin.conf
```

exportar la variable de entorno que le dice a kubctl donde esta su configuración

```
echo 'export KUBECONFIG=/etc/kubernetes/admin.conf' >> /root/.bashrc
source /root/.bashrc
```

verificar la configuración

```
kubectl get nodes
```

muestra algo como esto:

NAME STATUS ROLES AGE VERSION k8s-master01 NotReady control-plane 21m v1.28.15

Ya se puede volver a solicitar el token de unicón al cluster para usar en los nodos workers

```
kubeadm token create --print-join-command
```

mostrando algo como esto:

- kubeadm join k8s-master01:6443 --token o4cldfjlskjflkdsjflkdsjflkdsjflkdsjff2 --discovery-token-ca-cert-hash sha256:00fcf73bdc*

Copiar ese comando para ejecutarlo en el worker para que se una al custer

En el worker01 (192.168.122.196)

contando con la cadena para unirse al custer ejecutarla

```
kubeadm join k8s-master01:6443 --token sdssasa.1m15645054545d4f5d --discovery-token-ca-cert-hash
sha256:00fc39ie3209i0ie,a90zq9i09i2
```

acá se muestra ina cadena ficticia por seguridad

Pasar al master para verificar el resultado

En el master

Volver a listar los nodos

```
kubectl list nodes
```

probablemente muestre algun error, en ese caso reiniciarl el servicio

8. CNI Flannel

Flannel: Es

8.1 Instalar la red CNI de flannel

En el master01

Ejecuta

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml

despues
kubectl
```